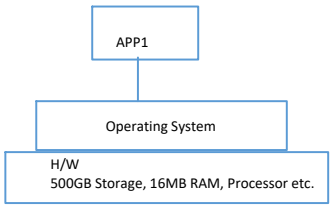
Docker & Kubernetes

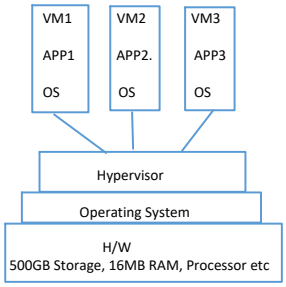
Sunday, 23 February 2025 3:51 PM

========== **Physical Servers, Virtual Servers & Containers** ==========

Physical Servers Limitations - Auto Scaling, Single OS, Can run single App only



Virtual Servers (example EC2 etc) - To overcome above limitations, Virtual Servers are introduced



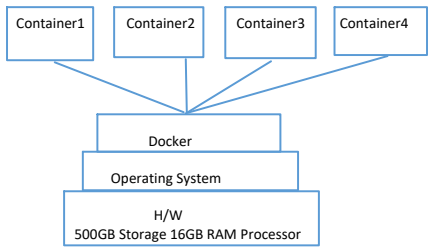
ınstall Hypervisor (software) to create multiple servers on the same H/W and different O/S (dedicated to each virtual server).
Hypervisor will distribute H/W over multiple virtual server and each virtual server will have dedicated O/S.
Virtual Servers can be created in on premise, AWS Cloud (EC2), Azure Cloud , Google Cloud (GCP) etc
Advantages - Multiple servers can be created with different O/S by distributing H/W on the same machine.

Docker (Container Tool) - Advantages over Virtual Servers

Dockers will be used instead Hypervisor in Containers to distribute H/W and O/P between the servers.
No dedicated O/S to containers
Docker will distribute both H/W and O/S where as in Virtual Servers only H/W will be distributed between Virtual Servers by Hypervisor. In place of VM, can create Docker container which is a complete server.
Can convert Container to Docker Image (will be a zip file, bundled with pre-requisites and app code) and image can be stored in registry which is **DockerHub**.

Advantages:

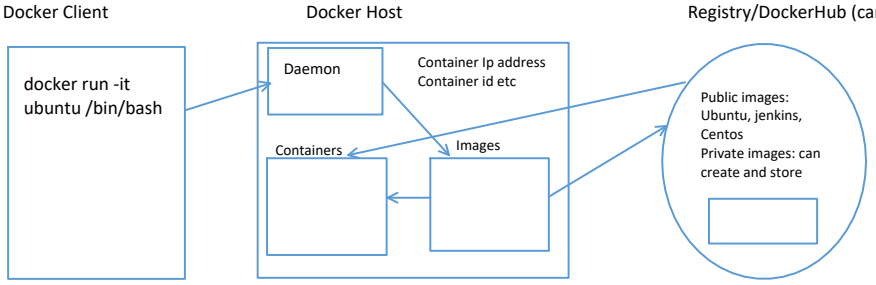
1. Can create containers (servers) using Docker Image easily and image will be automatically deployed in each container.
2. Auto scale up can be done easily using containers and easily destroy when the load is reduced.



3. Containers are light weight machine (OS will not be part of container) compare to VMs
4. Kubernetes will help to create multiple pods using the same docker image
5. Kubernetes will help to repair if any pod down
6. Zero downtime etc

In can base windows is OS in above infrastructure, any unix container can be created in above infrastructure since Docker comes with Kernel (which is a base for any unix). If base OS is UNIX, windows containers cannot be created since Docker has no window.

Docker Architecture



Usage	Docker Command
Docker Installation	Go to https://docs.docker.com/engine/install/ubuntu/ Execute below commands (refer https://docs.docker.com/engine/install/ubuntu/#install-using-the-convenience-script for commands) curl -fsSL https://get.docker.com -o get-docker.sh sudo sh get-docker.sh Executing docker install script, commit: 7cae5f8b0decc17d6571f9f52eb840fbc13b2737 <...>
Container creation	sudo docker run -it ubuntu /bin/bash ========== • **-i (interactive):** Keeps STDIN open so you can type commands. • **-t (pseudo-TTY):** Allocates a terminal interface. • **/bin/bash:** Opens the Bash shell.
list containers	sudo docker ps -a
To connect to docker container	sudo docker exec -it <container_name_or_id> /bin/bash
Remove docker container	docker rm <container_name_or_id>
To view docker images	docker images
To remove docker image	docker image rm <image name> or docker rmi <image-name>
To assign a specific name to a container while creating	docker run -it --name <container_name> ubuntu /bin/bash Example root@ip-172-31-30-52:/# docker run -it --name db-server1 ubuntu /bin/bash root@f141f6259dc7:/#
To assign a specific name to a container while creating and connect container with the same name instead ID	docker run -it -h db-server2 --name db-server2 ubuntu /bin/bash Example root@ip-172-31-30-52:/# docker run -it -h db-server2 --name db-server2 ubuntu /bin/bash root@db-server2: Note: -h stands for hostname
To start docker container	docker start <container_name_or_id>
To connect docker container	docker attach <contaiåner_name_or_id>
To start and connect docker container	docker start -ai <container_name_or_id>
To list docker containers in quite	docker ps -a -q
To delete all containers (combined command - remove and list) which are not up	docker rm $(docker ps -a -q)
	docker rm 'docker ps -a -q'
To list recent one container	docker ps -n1
To list recent 2 container	docker ps -n2
To list last created container	docker ps -l
To list running containers	docker ps
To list stopped containers	docker ps -a -f status=exited
To list the files/dirs of container from the host(VM or EC2 from where the container is created) without attaching to container	docker exec -it <container id or name> ls To be Tested : list for multiple containers using the same command
To create additional container and attach using single command	docker run -it -d ubuntu /bin/bash Note: 'd' means detach
Docker inspect command* - To check the actual creation date(timestamp), ip, storage, host machine, image id of the container - complete info	docker inspect <container id or name>
To check the ip address of the specific container	docker inspect --format {{.NetworkSettings.Networks.bridge.IPAddress}} <container id or name>
To check the ip address of all containers	docker inspect --format {{.NetworkSettings.Networks.bridge.IPAddress}} $(docker ps -a -q)
To login with name of the container	docker run -it --name sample -h sample ubuntu /bin/bash

Docker will have **docker container, docker image and docker volumes.**

Docker container command starts with **docker container** (command will work without container)

Docker container command starts with **docker container** (command will work without container)
Docker image command starts with **docker image**
Docker volumes command starts with **docker volumes**

DevOps is a culture (set of practices or processes) to automate all the stages using different tools (Git, Maven, Jenkins, Docker, Kubernetes, Terraform, Ansible etc) for early deployment with zero downtime

DevOps with AWS	Implementation of DevOps culture in Cloud using different tools Git, jenkins etc. Popular One
AWS(by Amazon) DevOps	AWS has introduced services (code commit, code build, code pipeline etc) for each tool to implement DevOps culture. Services are limited due to limitations
Azure(by Microsoft) DevOps	Azure has introduced their own services(Azure repos, Azure pipelines etc) to implement DevOps culture. Azure DevOps is more popular than AWS DevOps due to compatibility with Azure Cloud and unlimited services than AWS DevOps.

Cloud Providers - Amazon(AWS), Google(GCP), Microsoft(Azure), Oracle etc

Docker Image

In deploy an app in Virtual servers(ex: ec2), need to login to each server Individually and deploy the deploy.
To avoid using ECS2 and deploying in each server, we can use docker image/container.
We can create container using docker and deploy dockage image on it.
We can convert a docker container to docker image, OS,jdk,tomcat,app.war etc will be bundled and store in an image (which is a static and cannot start image but we can store) , this is called Application containerization

Command	Usage
docker run -it flipkarts-img /bin/bash	Create an image with the name flipkart-img

There are 2 Methods to create a docker image,

1. Commit method
   Test Case - create simple nginx image, nginx (is a server like tomcat, can deploy html files easily on ngiinx)
   nginx-cont



Create a container from ubuntu image - nginx-cont

```
docker run -it --name nginx-cont -p 8080:80 ubuntu /bin/bash   //To create a container and expose a port to access application from the browser
apt-get install nginx    //To install nginx in container
apt-get install vim    //To install vi editor
deploy index.html file under /var/www/html
<html>

  <body>

  <h1 style="color:red;">SR Consultancy Services</h1>

  </body>

</html>

service nginx start     //To start nginx
```

Check whether container is working or not

```
<public ip of the host>:8080              //access html via browser
```

Convert a container to an image

```
docker commit <container name> <DockerHub_User>/<image_name>     //To create an image with commit command(manually)
```

Test whether above image is working, create a container using above image access via browser

```
docker run -it --name nginx-cont1 -p 8080:80  9411/nginx-imgp101:latest /bin/bash   //create a container using above image  and expose port
```

Push above image to DockerHub to create container using above image in future

```
docker login -u 9144  //login to dockerhub
```

At the password prompt, enter PAT

```
dckr_pat_X9rTC738fBKiBmE43jB2an-8RGQ

docker push <DockerHub_User>/<image_name>
```

2. Build method, will be used in real time projects
   Need to write a Docker file for above steps

Dockerfile for above test case to create a container:

```
FROM ubuntu:22.04
RUN apt-get update
RUN apt-get install -y nginx
COPY index.html /var/www/html
ENTRYPOINT service nginx start && bash
```

```
docker build -t "9144/nginx-imagep101" .        //to create an image using above Dockerfile with build method
docker run -it --name nginx_container -p 8080:80 9144/nginx-imagep101 /bin/bash     //create a container and start nginx
```

http://44.192.41.83:8080/ //To test app via browser

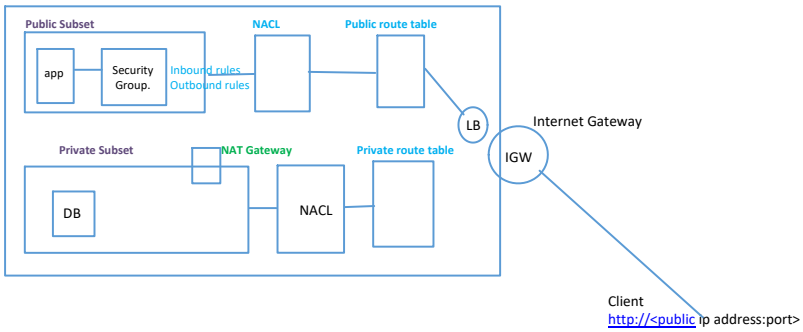
Kubernetes (Container Management Tool or Containerization)

Kubernetes has dependency on Docker image to create Pod

Virtual Private Cloud (VPC - private cloud within a Public Cloud)

To control the traffic routing from the same source
Multiple subnets can be connected to one subnet
One subnet cannot connect to multiple route tables

VPC 65000. Ips



Components in VPC
Subnets - Public subnet, Private subnet
NACLS
Security groups
Route Tables
Internet gateway
NAT Gateway

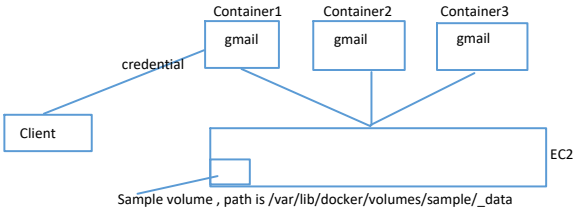
5772 - Default vpc

Docker Images Cont. - Create Docker Image with Tomcat

```
FROM ubuntu:22:04
RUN apt-get update
RUN apt-get install -y openjdk-21-jdk
ENV JAVA_HOME /usr
ADD apache-tomcat-11.0.26.tar.gz /root
COPY target/hello-maven.war /root/apache-tomcat-11.0.26/webapps
ENTRYPOINT /root/apache-tomcat-11.0.26/bin/startup.sh && bash
```

http://44.192.41.83:8090/hello-maven/

Docker Volumes - also called persistent volumes
To store data in other place which is related to container as data will wipeout when container goes down.
To share same data with multiple container



In above Test case, gmail is running in multiple containers. If 1 container goes down, client will get response from any other server which is up without any issues and without asking credentials again as credential info is in shared location which is accessible for all containers running on that particular host.

One way to maintain common data is in mount point which is not structured way.
Another approach is to create volume while creating container to store any such data and volume data will be stored in host machine.

Command	Usage
docker volume ls	To list the volumes in host
docker volume create <volume name>	To create the volume on host

docker inspect <volume name>	To check volume location on host
	
	o/p for volume - sample,
	root@ip-172-31-3-118:/home/ubuntu# docker inspect sample
	[
	{
	"CreatedAt": "2026-09-27T14:18:18Z",
	"Driver": "local",
	"Labels": null,
	"Mountpoint": "/var/lib/docker/volumes/sample/_data",
	"Name": "sample",
	"Options": null,
	"Scope": "local"
	}
	]

cd /var/lib/docker/volumes/sample/_data touch credentials	To create a file in above volume on host
docker run -it -v sample:/sample ubuntu /bin/bash	To create a container with above volume and mount volume to container Can view/edit volume from the container which is created in host
docker run -it --name container_volume13 -v sample:/sample:ro ubuntu /bin/bash	To create a container with volume read only mode to avoid updates from the container

Kubernetes (Container Management Tool or Containerization)

Containerization came into picture to avoid deployments separately in each virtual machines. To deploy app in Test env, need to do pre-requisites like OS, jdk, tomcat etc.
The same App may not run in some env's due to infra structure , version compatibility with VMs.

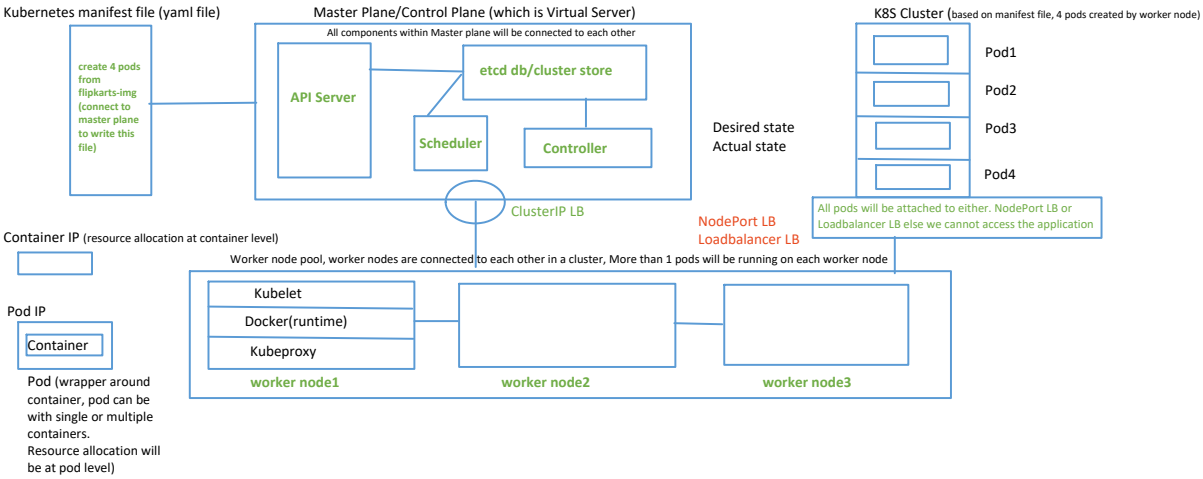
To avoid such issues, docker image came into picture, can create container using the same image and deploy app in any env.

Disadvantages in Docker
If the host(EC2) is down, containers created on that host will get impact. To over come this, create another EC2 and Maintain all hosts as cluster. This is called Docker Swarm concept.

Kubernetes will be used to manage containers hence Kubernetes is called container management or orchestration tool
Kubernetes is a cluster.

Kubernetes(K8s) Benefits
- Container management
- Can maintain zero downtime
- Auto scale up
- Self healing
- Rollback can be done easily through helm concept

2 Types of K8s
1. On premise K8s cluster - can setup K8s cluster on servers in on-premise Data Center or on any VMs in cloud
2. Managed K8s cluster - Cloud provider is providing Kubernetes cluster as a service which is managed Kubernetes (ready to use K8S clusters will be provided as service by cloud providers), mostly managed K8S cluster type will be used in real time.
   eg: EKS(AWS cloud - Elastic Kubernetes Service), GKE(Google Kubernetes Engine), AKS(Azure Kubernetes Service), OKE(Oracle Cloud Infrastructure - Oracle Kubernetes Engine)



http://<public ip of LB:port>/<end point of pod>

NodePort IP - free of cost. It has no dedicated IP, it will use any one of worker node IP address, it will automatically use another worker node IP if the current work node is down, it will use majorly in test environment
Loadbalancer IP - chargeable, will have dedicated IP, will be used in Prod environment
Cluster IP - will be created automatically

Container	Pod
App will run on container	Container will be inside Pod, app will run on container
Resource allocation at container level	Resource allocation at Pod level
	Within a Pod, containers will be connected using Port Mapping method
Single container	Can have 1 or more containers in each Pod

Scheduler will identity the load on each worker node using Cluster IP Type of LB service
Based on the load, Schedule will give the request to Kubelet of the particular worker node to create a Pod.
Kubelet will create pods in a K8S cluster with the help of Docker based on the request received from Scheduler.
Kubeproxy will allocate IP addresses to each Pod once created

Controller will set Desired state = number of pods in a cluster when pods created. Controller will continuously check the actual state of a Pod in a cluster, if the Actual state count is less than desired state count of the pods in a cluster, Kubernetes will check and bring up the pod which is not up.

Note:- API Server, etcd db/cluster store etc are containers hence cannot use another Container as Control Plane as we cannot have container inside container.

Popular methods to create on-premise Kubernetes cluster,
1. Kind method - Used primarily for local development and testing.
2. Kubeadm method - this will be the most commonly used method, especially for setting up production clusters
3. Minikube method - Creates both the control/master plane and worker node within the same cluster. This method is intended for development/testing and will not be used in production.

On-premise cluster - both Control Plane and Worker Node using kubeadm Method, will not be used in prod due to limitations and self managed cluster

Steps to Create Control Plane and make it ready			
S.No	Command	Usage	Remarks
1		Create ec2	t3.micro may not support for Control Plane due to space constraint, select c7i-flex-large as instance type
2		Install Docker	curl -fsSL https://get.docker.com -o get-docker.sh sudo sh get-docker.sh Executing docker install script, commit: 7cae5f8b0decc17d6571f9f52eb840fbc13b2737 <...>
3		Install Java	For only kubernetes cluster, no need java but it is required to run yaml files
4		Install kubeadm	Refer https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/install-kubeadm/ link for the steps to install kubelet kubeadm kubectl
5		Install kubelet	
6	kubeadm init	Initiate kubeadm - To create Control plane with all components	
7		Delete toml flie	To the fix the error encountered during initiate kubeadm /etc/containerd/config.toml

			/etc/containerd/config.toml
8	systemctl restart containerd	Restart containerd	After redeleting toml file, restart containerd before reset and reinitiating kubeadm
	~~kubeadm reset~~	~~Reset kubeadm~~	
		~~Re Initiate kubeadm~~	
	~~systemctl restart containerd~~	~~Restart containerd~~	~~If any error during re initiate kubeadm~~
9	kubeadm reset		
10	kubeadm init		Your Kubernetes control-plane has initialized successfully! To start using your cluster, you need to run the following as a regular user: mkdir -p $HOME/.kube sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config sudo chown $(id -u):$(id -g) $HOME/.kube/config Alternatively, if you are the root user, you can run: export KUBECONFIG=/etc/kubernetes/admin.conf You should now deploy a pod network to the cluster. Run "kubectl apply -f [podnetwork].yaml" with one of the options listed at: https://kubernetes.io/docs/concepts/cluster-administration/addons/ Then you can join any number of worker nodes by running the following on each as root: kubeadm join 172.31.20.131:6443 --token oa576v.73y4012957cqlg3m \ --discovery-token-ca-cert-hash sha256:2e7f24f36b13b2768d6b24ee1c0ca85eb7eec93e266969a699517e9f48562294 root@ip-172-31-20-131:/#
11	mkdir -p $HOME/.kube sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config sudo chown $(id -u):$(id -g) $HOME/.kube/config	Run these 3 command (from the above O/P) 1 by 1 in sequence to make EC2 instance as Control Plane	
12	kubectl get nodes	To check the status of control pane	O/P NAME STATUS ROLES AGE VERSION ip-172-31-17-61 NotReady control-plane 7m19s v1.37.1
13	kubectl get namespaces	To check namespace	O/P root@ip-172-31-17-61:/# kubectl get namespaces NAME STATUS AGE default Active 10m kube-node-lease Active 10m kube-public Active 10m kube-system Active 10m root@ip-172-31-17-61:/#
14	kubectl get pods --all-namespaces	To list the pods in Control Plane	O/P root@ip-172-31-17-61:/# kubectl get pods --all-namespaces NAMESPACE NAME READY STATUS RESTARTS AGE kube-system coredns-559f6c778d-49gqj 0/1 Pending 0 16m kube-system coredns-559f6c778d-w7d68 0/1 Pending 0 16m kube-system etcd-ip-172-31-17-61 1/1 Running 0 16m kube-system kube-apiserver-ip-172-31-17-61 1/1 Running 0 16m kube-system kube-controller-manager-ip-172-31-17-61 1/1 Running 0 16m kube-system kube-proxy-65dkj 1/1 Running 0 16m kube-system kube-scheduler-ip-172-31-17-61 1/1 Running 0 16m root@ip-172-31-17-61:/#
15	kubectl apply -f https://reweave.azurewebsites.net/k8s/v1.29/net.yaml	To establish network connection for top2 nodes (coredns)	To establish network connection for top 2 components in above O/P Go to https://kubernetes.io/docs/concepts/cluster-administration/addons/ (link is in #10) Select open Weave Net in https://github.com/rajch/weave#using-weave-on-kubernetes In**Using Weave Net on Kubernetes**, On a newly created Kubernetes cluster, the Weave Net CNI pluging can be installed by running the following command: kubectl apply -f https://reweave.azurewebsites.net/k8s/v1.29/net.yaml O/P oot@ip-172-31-17-61:/# kubectl apply -f https://reweave.azurewebsites.net/k8s/v1.29/net.yaml serviceaccount/weave-net created clusterrole.rbac.authorization.k8s.io/weave-net created clusterrolebinding.rbac.authorization.k8s.io/weave-net created role.rbac.authorization.k8s.io/weave-net created rolebinding.rbac.authorization.k8s.io/weave-net created daemonset.apps/weave-net created root@ip-172-31-17-61:/#
16	kubectl get pods --all-namespaces	To check the status of pods in control panel	root@ip-172-31-17-61:/# kubectl get pods --all-namespaces NAMESPACE NAME READY STATUS RESTARTS AGE kube-system coredns-559f6c778d-49gqj 1/1 Running 0 33m kube-system coredns-559f6c778d-w7d68 1/1 Running 0 33m kube-system etcd-ip-172-31-17-61 1/1 Running 0 34m kube-system kube-apiserver-ip-172-31-17-61 1/1 Running 0 34m kube-system kube-controller-manager-ip-172-31-17-61 1/1 Running 0 34m

			kube-system kube-proxy-65dkj 1/1 Running 0 33m kube-system kube-scheduler-ip-172-31-17-61 1/1 Running 0 34m kube-system weave-net-6fkst 2/2 Running 1 (7m55s ago) 7m58s root@ip-172-31-17-61:/#
17	kubectl get nodes	To check the status of control plane	oot@ip-172-31-17-61:/# kubectl get nodes NAME STATUS ROLES AGE VERSION ip-172-31-17-61 Ready control-plane 35m v1.37.1 root@ip-172-31-17-61:/#

Steps to setup a worker node (with single node, same process for multi node also)

S.No	Command	Usage	Remarks
1	Create a worker node	Repeat steps from (1) to (10) under Control Plane	
2	Take similar join command below from kubeadm init O/P in Control plane and run on worker node Example: kubeadm join 172.31.17.61:6443 --token oeggqp.kdx1d80f24rka8i3 \ --discovery-token-ca-cert-hash sha256:67bc138ff8f09412203c6106278c6450eb28acc50f7e6962487b30dfc880e3d7	To connect worker node to Control Plane	
3	systemctl restart containerd	If any ERROR in #2, restart containerd	
4	kubeadm reset	Reset kubeadm	
5	Rerun join command #2	Re run join command after kubeadm reset	
6	systemctl restart kubelet	If any error in #5, restart kubelet	
7	systemctl restart containerd	Restart containerd	
8	kubeadm reset		
9	Rerun join command from #2		
10	If any issues in #9, repeat steps from #6 to #9		

Managed K8S Cluster - Steps to create Master Plane and Worker Nodes, prerequisites to create EKS cluster

S.No	Command	Usage	Remarks
1	Login to AWS --> search IAM --> look for IAM users in IAM dashboard --> Select IAM Users under left menu --> In Specify user details screen, enter user name then hit Next button -->> In Set permissions, select Attach policies directly then select AdministratorAccess , hit Next --> In review and create, hit create user	Create IAM User via AWS console	This is mandatory for EKS
2	Select user --> select security credentials --> Under Access Key, select create Access Key --> Select "Command Line Interface" use case , select checkbox in the bottom then hit create access key --> Both Access and Secret Key will get create successfully. Copy keys and save somewhere as both these Keys will be used during EKS setup.	Generate Access and Secret key for IAM user created in #1	
3	Create EC2 instance via AWS console		
4	apt-get update apt-get install unzip	Install unzip (command required for AWS cli installation)	
5	Under AWS CLI install instructions --> Linux --> go to tab ***Command line installer - Linux X86(64 bit),*** *copy below command execute all commands in one go (if unzip command* is already installed) curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip" unzip awscliv2.zip sudo ./aws/install	Install AWS CLI In browser, search **iinstall aws cli on ubuntu** https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html	
6	root@ip-172-31-14-31:/# aws --version aws-cli/2.37.6 Python/3.14.6 Linux/7.0.0-1006-aws exe/x86_64.ubuntu.26 root@ip-172-31-14-31:/#	Check AWS CLI cersion	
7	root@ip-172-31-14-31:/# aws configure Tip: You can deliver temporary credentials to the AWS CLI using your AWS Console session by running the command 'aws login'. AWS Access Key ID [None]: AKIAYBN74MWQLW5ZUD2T AWS Secret Access Key [None]: G318FARMRB9SbYXESkD1qHjaMKNiCN4U0w2OzAA4 Default region name [None]: us-east-1 Default output format [None]: json root@ip-172-31-14-31:/#	Configure AWS CLI with the credentials created in step #2	
8	Step #1 curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl" Step #2 curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl.sha256" Step #3 sudo install -o root -g root -m **0755** kubectl /usr/local/bin/kubectl Step #4 root@ip-172-31-14-31:/# **kubectl version --client** Client Version: v1.37.1 Kustomize Version: v5.8.1 root@ip-172-31-14-31:/#	**kubectl Installation** EC2 created in #3 works as Master/Control plane, , need to install kubectl to run kubernetes commands from Master Plane In browser, search **install kubectl on ubuntu** https://kubernetes.io/docs/tasks/tools/install-kubectl-linux/	

9	root@ip-172-31-14-31:/# # for ARM systems, set ARCH to: `arm64`, `armv6` or `armv7`
	ARCH=$(uname -m | sed 's/x86_64/amd64/' | sed 's/aarch64/arm64/')
	PLATFORM=$(uname -s)_$ARCH
	
	curl -sLO "https://github.com/eksctl-io/eksctl/releases/latest/download/eksctl_$PLATFORM.tar.gz"
	
	# (Optional) Verify checksum
	curl -sL "https://github.com/eksctl-io/eksctl/releases/latest/download/eksctl_checksums.txt" | grep $PLATFORM | sha256sum --check
	
	tar -xzf eksctl_$PLATFORM.tar.gz -C /tmp && rm eksctl_$PLATFORM.tar.gz
		EKSCTL Installation - Required to run eks commands to create EKS cluster from Master Plane
		
		In browser, search **install eksctl on ubuntu**
		Select GitHub link, https://github.com/eksctl-io/eksctl
		
		Under **For Unix,** copy and run command in EC2	

	sudo install -m 0755 /tmp/eksctl /usr/local/bin && rm /tmp/eksctl eksctl_Linux_amd64.tar.gz: OK root@ip-172-31-14-31:/#		
10	root@ip-172-31-14-31:/# eksctl version 0.230.0 root@ip-172-31-14-31:/#	To check eksctl version	
11	eksctl create cluster --name eks-m102 --region us-east-1 --nodegroup-name standard-workers --node-type t3.small --nodes 3 --nodes-min 1 --nodes-max 3 eksctl create cluster --name eks-m102 --region us-east-1 --nodegroup-name standard-workers --node-type t3.micro --nodes-min 1 --nodes-max 3 Run below command to scale your existing node group to 3 nodes, eksctl scale nodegroup --cluster eks-m102 --name standard-workers --nodes 3 --nodes-min 1 --nodes-max 3 --region us-east-1	****Command to create EKS cluster	
12	root@ip-172-31-14-31:/# eksctl get cluster NAME REGION EKSCTL CREATED eks-m102 us-east-1 True root@ip-172-31-14-31:/#	Command to check the cluster details	
13	root@ip-172-31-14-31:/# aws eks update-kubeconfig --name eks-m104 --region us-east-1 Added new context arn:aws:eks:us-east-1:552842585504:cluster/eks-m102 to /root/.kube/config root@ip-172-31-14-31:/#	Configure node to region	
14	root@ip-172-31-28-156:/# kubectl get nodes NAME STATUS ROLES AGE VERSION ip-192-168-25-168.ec2.internal Ready <none> 8m51s v1.34.11-eks-f4fc4f1 ip-192-168-25-194.ec2.internal Ready <none> 8m50s v1.34.11-eks-f4fc4f1 ip-192-168-40-144.ec2.internal Ready <none> 8m50s v1.34.11-eks-f4fc4f1	To check the worker pool	
15	Once command completed in #14, go to EKS in dashboard to check how many nodes created, check the node groups under compute tab		
16	Post validation, release all resources to avoid charges 1. Delete nodes under EKS --> compute 2. Delete cluster eks-m102 3. Delete ec2-eks 4. Delete NAT gateway 5. Delete cluster specific VPC 6. Release Elastip IP 7. Delete stacks under natcloudscloud formation Command to delete resources including cluster in backend, eksctl delete cluster --name eks-m102 --region us-east-1	Cean-up to avoid charges	
17	kubectl run nginx-pod --image=nginx --restart=Never	To create single pod	
18	root@ip-172-31-28-156:/# kubectl get pods -o wide NAME READY STATUS RESTARTS AGE IP NODE NOMINATED NODE READINESS GATES nginx-pod 1/1 Running 0 61s 192.168.54.174 ip-192-168-40-144.ec2.internal <none> <none>	Check the pod O/P	
19	kubectl describe pod nginx-prod	Describe the pod, pod is created on default node	
20	kubectl get nodes Kubectl get pods root@ip-172-31-28-156:/# cat nginx-pod.yaml apiVersion: v1 kind: Pod metadata: name: nginx-pod labels: env:prod version:1.2.3 spec: nodeName: **ip-192-168-25-168.ec2.internal** containers: - name: nginx-container image: nginx ports: - containerPort: 80 rroot@ip-172-31-28-156:/# kubectl apply -f nginx-pod.yaml pod/nginx-pod created root@ip-172-31-28-156:/# kubectl get pod nginx-pod -o wide NAME READY STATUS RESTARTS AGE IP NODE NOMINATED NODE READINESS GATES nginx-pod 1/1 Running 0 15s 192.168.2.38 ip-192-168-25-168.ec2.internal <none> <none> root@ip-172-31-28-156:/# Command to delete a pod Kubectl delete pod nginx-pod	If you specifically want the pod on a particular worker node First list your nodes:	
21	root@ip-172-31-28-156:/# vi nginx-3pods.yaml root@ip-172-31-28-156:/# kubectl apply -f nginx-3pods.yml pod/nginx-pod-1 created pod/nginx-pod-2 created pod/nginx-pod-3 created root@ip-172-31-28-156:/# kubectl get pods -o wide NAME READY STATUS RESTARTS AGE IP NODE NOMINATED NODE READINESS GATES nginx-pod 1/1 Running 0 4m35s 192.168.2.38 ip-192-168-25-168.ec2.internal <none> <none> nginx-pod-1 1/1 Running 0 16s 192.168.54.174 ip-192-168-40-144.ec2.internal <none> <none> nginx-pod-2 1/1 Running 0 16s 192.168.27.15 ip-192-168-25-194.ec2.internal <none> <none> nginx-pod-3 1/1 Running 0 16s 192.168.60.192 ip-192-168-40-144.ec2.internal <none> <none> root@ip-172-31-28-156:/#	nginx-3pods.yaml ========== apiVersion: v1 kind: Pod metadata: name: nginx-pod-1 spec: containers: - name: nginx image: nginx:latest --- apiVersion: v1 kind: Pod metadata: name: nginx-pod-2 spec: containers: - name: nginx image: nginx:latest --- apiVersion: v1 kind: Pod metadata: name: nginx-pod-3 spec: containers: - name: nginx image: nginx:latest	
22	kubectl delete pods --all	To remove pods	

	kubectl delete pod nginx-pod nginx-pod-1 nginx-pod-2 nginx-pod-3 **Self healing capability is not applicable** if we create a pod using above pod method. To overcome this, deployment method type will be used for creating pods (refer #25 for deployment.yaml file and commands to create pod creating).		
23	eksctl delete cluster --name eks-m104 --region us-east-1	To remove EKS cluster and its managed resources created by eksctl. This is the main cleanup command. It will delete the EKS cluster, managed node group/worker nodes, and the CloudFormation stacks/resources that eksctl created for the cluster, including the VPC-related resources it owns.	
24	Check that the cluster is gone: aws eks describe-cluster \ --name eks-m104 \ --region us-east-1 Check CloudFormation: aws cloudformation list-stacks \ --region us-east-1 \ --stack-status-filter CREATE_COMPLETE UPDATE_COMPLETE UPDATE_ROLLBACK_COMPLETE Check EC2 resources: aws ec2 describe-nat-gateways \ --region us-east-1 \ --filter Name=state,Values=available,pending Check Elastic IPs: aws ec2 describe-addresses \ --region us-east-1 If you want to see what eksctl created Before deleting, you can run: eksctl utils describe-stacks \ --cluster eks-m104 \ --region us-east-1		
25	To check the list of nodes **Kubectl get pods** **Deployment.yaml** apiVersion: apps/v1 kind: Deployment metadata: name: nginx-prod-deploy Labels: app: nginx-app-prod spec: replicas: 3 selector: matchLabels: app: nginx-pod template: metadata: labels: app: nginx-prod spec: containers: - name: nginx-container image: nginx ports: - containerPort: 80 **Note:** app highlighted (in yellow) in above deployment.yaml should match Run below command to create 6 pods using above deployment.yaml file kubectl apply -f deployment.yaml Check the Deployment and Pod: kubectl get deployments kubectl get pods Delete pod to test self healing capability kubectl delete <pod-name> Check whether new pod gets created automatically kubectl get pods To check namespace details, kubectl get namespace kubectl create namespace		
	kubectl	To delete pod permanently	
	service.yaml apiVersion: v1 kind: Service metadata: name:nginx-prod-service labels: app: nginx-app-prod-service spec: selector: app: nginx-prod type: LoadBalancer ports:	Create LoadBalancer LB using service manifest file	

	ports: - port: 80 targetPort: 80 Command to run service yaml file, root@ip-172-31-13-123:/home/ubuntu/deployments# kubectl apply -f service.yaml service/nginx-prod-service created		
	To check cluster IP type of load balancer kubectl get services	Kubernetes created cluster IP service automatically to distribute the load on worker nodes	
	kubectl describe svc <service-name> root@ip-172-31-13-123:/home/ubuntu/deployments# kubectl describe svc nginx-prod-service Name: nginx-prod-service Namespace: default Labels: app=nginx-app-prod-service Annotations: <none> Selector: app=nginx-prod Type: LoadBalancer IP Family Policy: SingleStack IP Families: IPv4 IP: 10.100.54.89 IPs: 10.100.54.89 LoadBalancer Ingress: ae796ea6bf40c4c61b08c75bc7413da2-647256705.us-east-1.elb.amazonaws.com Port: <unset> 80/TCP TargetPort: 80/TCP NodePort: <unset> 32739/TCP Endpoints: **192.168.33.57**:80,**192.168.24.73**:80,**192.168.55.97**:80 Session Affinity: None External Traffic Policy: Cluster Internal Traffic Policy: Cluster Events: Type Reason Age From Message ---- ------ ---- ---- ------- Normal EnsuringLoadBalancer 12m service-controller Ensuring load balancer Normal EnsuredLoadBalancer 12m service-controller Ensured load balancer	To check whether service tagged to deployment	
	kubectl get pods -o wide root@ip-172-31-13-123:/home/ubuntu/deployments# kubectl get pods -o wide NAME READY STATUS RESTARTS AGE IP NODE NOMINATED NODE READINESS GATES nginx-prod-deploy-5d5d495c7c-dj544 1/1 Running 0 32m **192.168.24.73** ip-192-168-16-255.ec2.internal <none> <none> nginx-prod-deploy-5d5d495c7c-hzmj2 1/1 Running 0 32m **192.168.33.57** ip-192-168-49-193.ec2.internal <none> <none> nginx-prod-deploy-5d5d495c7c-rrdwl 1/1 Running 0 32m **192.168.55.97** ip-192-168-55-80.ec2.internal <none> <none> root@ip-172-31-13-123:/home/ubuntu/deployments#	To check wide output of pods	

1. Clone repo: git clone https://github.com/manjulagali/flipkarts.git
2. Build using Maven with pom.xml. -
   install jdk - apt-get install openjdk-21-jdk
   Install Maven: apt-get install maven
   Build flipkarts (inside flipkarts folder): mvn install
3. Create Image for above build using Docker
   Install Docker: Refer Docker section for command
   Docker Image: docker build -t "9144/flipkarts102-img" . // . for current dir
   List images:
   root@ip-172-31-13-123:/home/ubuntu/source/flipkarts# docker images

   i Info → U In Use

   IMAGE ID DISK USAGE CONTENT SIZE EXTRA
   9144/flipkarts102-img:latest dca45762ccb7 1.27GB 377MB

   Push image to DockerHub,
   A. Login to docker hub: docker login -u 9144
   B. Push Image: docker push 9144/flipkarts102-img:latest
4. Delete deployments and LB Service from kubernetes if any

   oot@ip-172-31-13-123:/home/ubuntu/source/flipkarts# kubectl get deployments
   NAME READY UP-TO-DATE AVAILABLE AGE
   nginx-prod-deploy 3/3 3 3 3h47m
   root@ip-172-31-13-123:/home/ubuntu/source/flipkarts# kubectl delete deployment nginx-prod-deploy
   deployment.apps "nginx-prod-deploy" deleted from default namespace

   root@ip-172-31-13-123:/home/ubuntu/source/flipkarts# kubectl get service
   NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
   kubernetes ClusterIP 10.100.0.1 <none> 443/TCP 4h
   nginx-prod-service LoadBalancer 10.100.54.89 ae796ea6bf40c4c61b08c75bc7413da2-647256705.us-east-1.elb.amazonaws.com 80:32739/TCP 3h30m
   root@ip-172-31-13-123:/home/ubuntu/source/flipkarts# kubectl delete svc nginx-prod-service
   service "nginx-prod-service" deleted from default namespace

5. Deploy Docker Image using deployment.yml and create service using service.yml in kubernetes
   Create deployment1.yaml file
   root@ip-172-31-13-123:/home/ubuntu/deployments# kubectl apply -f deployment1.yaml
   deployment.apps/flipkarts-deploy created
   root@ip-172-31-13-123:/home/ubuntu/deployments# kubectl get deployments
   NAME READY UP-TO-DATE AVAILABLE AGE
   flipkarts-deploy 3/3 3 3 7s
   root@ip-172-31-13-123:/home/ubuntu/deployments# kubectl get pods
   NAME READY STATUS RESTARTS AGE
   flipkarts-deploy-65d9c9d5d8-fgm8m 1/1 Running 0 17s
   flipkarts-deploy-65d9c9d5d8-kb5tq 1/1 Running 0 17s
   flipkarts-deploy-65d9c9d5d8-kgrx2 1/1 Running 0 17s

6. Create service yaml file for Loadbalancer LB
   Create service1.yaml

   root@ip-172-31-13-123:/home/ubuntu/deployments# kubectl get service
   NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
   kubernetes ClusterIP 10.100.0.1 <none> 443/TCP 4h42m

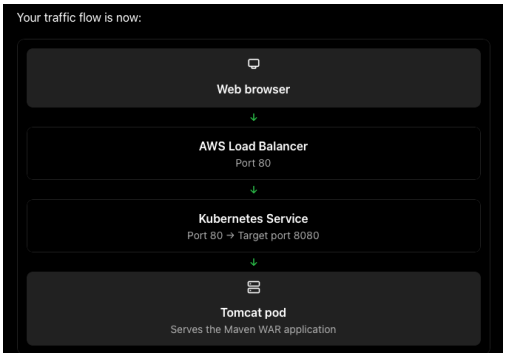
   root@ip-172-31-13-123:/home/ubuntu/deployments# kubectl apply -f service1.yaml
   service/flipkarts-service created

service/flipkarts-service created

root@ip-172-31-13-123:/home/ubuntu# kubectl get service

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
flipkarts-service	LoadBalancer	10.100.224.86	a38f71345b9b24db485b82167ee981f4-1748092523.us-east-1.elb.amazonaws.com	80:32649/TCP	7m35s
kubernetes	ClusterIP	10.100.0.1	<none>	443/TCP	5h19m

Access application in browser using URL - <LB url>/hello-maven



Next Topics

1. Rollback using Helm
2. Auto scaling. - Increase load artificially to check whether number of pods getting increased or not as part of Auto scaling.
   Auto scaling methods
3. Rback - used for security
4. Terraform
5. AWS additional concepts